

Práctica de Laboratorio 1

Stack y Queue: especificación, memoria, STL y costo

Algoritmos y Estructuras de Datos

Propósito

En teoría distinguimos el **tipo abstracto de datos (TDA)**, que especifica las operaciones y su comportamiento, de la **estructura de datos**, que decide cómo se representan los datos y determina el costo de esas operaciones.

En esta práctica implementará distintas representaciones de **Stack** y **Queue**. Cada implementación deberá superar primero el **mismo sistema de pruebas de correctitud**; solo después podrá entrar al benchmark. El objetivo es observar que dos implementaciones pueden ser igualmente correctas y tener costos radicalmente diferentes.

La práctica sirve además como repaso de C++: punteros, memoria dinámica, ownership, copia profunda, semántica de movimiento, regla de cinco, regla de cero, plantillas, STL y búsqueda binaria.

Objetivos

Al finalizar deberá ser capaz de:

- administrar correctamente un bloque de memoria contigua;
- implementar copia profunda y transferencia de ownership mediante movimiento;
- relacionar propiedades del TDA con casos de prueba;
- usar STL como implementación de referencia y como oráculo de pruebas;
- explicar por qué una representación correcta puede resultar ineficiente;
- implementar y medir búsqueda lineal frente a búsqueda binaria;
- interpretar evidencia experimental junto con el análisis asintótico.

Proyecto base

Se trabaja con C++20. En Ubuntu/Debian:

```
sudo apt update
sudo apt install g++ make doctest-dev
```

El proyecto contiene:

```
include/
  stack_raw.hpp
  stack_vector.hpp
  queue_shift.hpp
  queue_circular.hpp
  bounds.hpp
```

```
tests/  
    test_main.cpp  
    test_structures.cpp  
bench/  
    benchmark.cpp  
Makefile
```

Use durante el desarrollo:

```
make test
```

Los tests se compilan con AddressSanitizer y UndefinedBehaviorSanitizer. Cuando todos pasen:

```
make bench
```

o simplemente:

```
make run
```

El benchmark solo debe interpretarse si las pruebas de correctitud han pasado.

1. Stack con memoria administrada manualmente

Complete `StackRaw<T>` usando un arreglo dinámico contiguo:

```
T *data_  
std::size_t size_  
std::size_t capacity_;
```

Para mantener el foco en ownership y representación, esta implementación didáctica puede asumir que `T` es construible por defecto y asignable.

Nota: esta es una simplificación didáctica. Las pruebas usan principalmente tipos sencillos como `int`. Un contenedor genérico de producción debe gestionar además la construcción y destrucción individual de objetos no triviales; ese problema queda fuera del alcance de esta práctica.

Debe implementar:

- `push`, `pop`, `top`, `empty` y `size`;
- crecimiento geométrico cuando el arreglo se llena;
- destructor;
- constructor y asignación por copia;
- constructor y asignación por movimiento.

La copia debe ser profunda. El movimiento debe transferir el recurso sin copiar todos los elementos y dejar el origen **vacío y destruible**.

Contrato

`top()` y `pop()` sobre una pila vacía deben lanzar `std::out_of_range`.

Los costos esperados son:

Operación	Costo
push	$\mathcal{O}(1)$ amortizado
pop	$\mathcal{O}(1)$
top	$\mathcal{O}(1)$
empty, size	$\mathcal{O}(1)$

Propiedades usadas por los tests

El sistema de pruebas comprueba, entre otras:

$$\text{empty}(\text{new}) = \text{true}, \quad \text{size}(\text{new}) = 0$$

$$\text{top}(\text{push}(s, x)) = x$$

$$\text{size}(\text{push}(s, x)) = \text{size}(s) + 1$$

y múltiples secuencias pseudoaleatorias comparadas con `std::stack<int>`.

No modifique los tests para ocultar un error de implementación.

2. La misma pila mediante STL

Estudie `StackVector<T>`, que usa:

```
std::vector<T> data_;
```

La interfaz observable es esencialmente la misma, pero la clase ya no administra directamente un bloque de memoria. Observe por qué desaparece la necesidad de escribir destructor, constructor de copia, constructor de movimiento y operadores de asignación: es un ejemplo de **regla de cero**.

Responda brevemente:

- ¿qué responsabilidad ha sido delegada a `std::vector`?
- ¿qué parte del TDA permanece idéntica?
- ¿por qué usar STL no elimina la necesidad de entender el costo de las operaciones?

3. Queue con desplazamiento

Implemente `QueueShift<T>` manteniendo siempre el frente en `data_[0]`.

Si tenemos:

```
[10] [20] [30] [40]
```

después de `pop()` la representación debe quedar:

```
[20] [30] [40] [ ]
```

por lo que deberá desplazar los elementos restantes.

La clase también administra memoria manualmente y debe respetar la regla de cinco. `front()` y `pop()` sobre una cola vacía deben lanzar `std::out_of_range`.

La variable `moves_` cuenta cada transferencia de un elemento causada por:

- un redimensionamiento;
- un desplazamiento al eliminar el frente.

No cuente la asignación normal del nuevo elemento en `push`.

Antes de medir, complete:

Operación	Costo esperado	Motivo
<code>push</code>		
<code>front</code>		
<code>pop</code>		

4. Queue circular: mismo TDA, otra representación

Implemente ahora `QueueCircular<T>` mediante un buffer circular. El elemento lógico i puede encontrarse con una expresión basada en:

$$(head + i) \bmod capacity.$$

Requisitos:

- `pop()` no debe desplazar todos los elementos;
- `front()` debe ser $\mathcal{O}(1)$;
- `push()` debe ser $\mathcal{O}(1)$ amortizado;
- al crecer, el nuevo buffer debe preservar el orden lógico de la cola;
- debe superar el mismo tipo de pruebas que `QueueShift`;
- debe soportar correctamente wrap-around, copia y movimiento.

Esta es la idea central del ejercicio: **el TDA no cambió; cambió la representación.**

5. Correctitud y desempeño en el mismo experimento

El programa de pruebas aplica operaciones aleatorias simultáneamente a sus colas y a `std::queue<int>`. Después de cada operación compara tamaño, estado vacío y frente.

Solo si las pruebas pasan, ejecute:

```
make bench
```

El benchmark vuelve a comprobar que las implementaciones produzcan el mismo resultado mediante un checksum y aborta si detecta una discrepancia. Además compara:

- `QueueShift`;
- `QueueCircular`;
- `std::queue`.

Para cada n realiza n inserciones y luego elimina todos los elementos. Además del tiempo, las implementaciones propias reportan el número de transferencias estructurales realizadas.

Prediga primero cómo crecerán los movimientos de `QueueShift` y `QueueCircular`. Después contraste su predicción con los resultados medidos.

6. Implementar `lower_bound` y `upper_bound`

Complete:

```
template<typename T>
std::size_t my_lower_bound(const std::vector<T> &a, const T &x);

template<typename T>
std::size_t my_upper_bound(const std::vector<T> &a, const T &x);
```

Puede asumir que `a` está ordenado.

La semántica requerida es:

- `lower_bound`: primera posición i tal que $a[i] \geq x$;
- `upper_bound`: primera posición i tal que $a[i] > x$.

Si no existe tal posición, devuelva `a.size()`.

Para:

1 2 2 2 4 7 7 9

se espera:

`lower_bound(2) = 1`

`upper_bound(2) = 4`

No se permite búsqueda lineal. Las pruebas verifican resultados contra `std::lower_bound`/`std::upper_bound`, y la evaluación adicional comprueba que el número de comparaciones sea compatible con $\mathcal{O}(\log n)$.

El benchmark compara además una búsqueda lineal, su `my_lower_bound` y `std::lower_bound` para muchas consultas.